



Piano keyboard detection algorithm using deep learning

Tserenpurev Chuluunsaikhan¹ · So-Hyun Park² 

Received: 12 January 2026 / Revised: 10 April 2026 / Accepted: 27 April 2026

© The Author(s), under exclusive licence to Springer Science+Business Media, LLC, part of Springer Nature 2026

Abstract

In this study, a piano keyboard detection approach was investigated. Because accurate recognition of a piano keyboard is linked to the accuracy of music transcription, piano keyboard detection is a fundamental step in research on music transcription and is a critical problem. Previous studies on piano keyboard detection have mainly focused on videos taken within a limited environment or taken above the piano. However, actual piano playing videos are often captured not only in limited environments, but also in various performance venues. Furthermore, in addition to filming above the piano, the piano is often filmed from the side. Using such videos capturing actual piano playing, it is difficult to accurately detect the piano keyboard through conventional computer vision technology. To address this challenge, this study introduces the Image Combining Method using Partial Pixel Information (ICM_PixInfo), which combines the partial pixel data of multiple images to enhance the accuracy of piano keyboard detection. Additionally, this study proposes the PD_BiTh algorithm, which extracts piano key positions from combined images. The performance of the proposed PiKd was compared with that of conventional methods. The results show that PiKd achieves the highest recall, reaching approximately 89.33%, and outperforms the comparative methods by about 21.05%p to 41.25%p on average.

Highlights

- We introduce the ICM_PixInfo that enhances detection accuracy by combining partial pixel data from multiple images. The method identifies the image with the fewest hand-colored pixels as a reference, removes detected hand regions from this image, and synthesizes a new image by combining it with other frames. As a result, the final image significantly reduces the influence of hand-related noise, improving the visibility and detection of piano keys.
- We propose the Polygon Detection with Binary Threshold algorithm (PD_BiTh) to extract piano key positions from the combined image. PD_BiTh detects a polygon by adjusting the threshold value and the degree of shape expansion when binarizing the image. Then, the Propriety Test of Degree of Expansion algorithm (PT_ExpDg) is applied to inspect the adequacy of the expansion, and the Polygon Simplification Algorithm (PSAI) algorithm is used for polygon simplification. Finally, Substitution Algorithm (SubAI) is used, which applies the piano keyboard coordinates detected to every individual image making up a combined image.

Extended author information available on the last page of the article

- We demonstrate the practical effectiveness of PiKd using the C3Pap dataset. We compare its performance with that of conventional methods in terms of recall at different overlap areas between piano keys and human hands, specifically at 20%, 40%, 60%, and 80%. The results show that PiKd consistently outperforms conventional methods such as CV_Poly and CV_Square in terms of recall, highlighting its robustness and accuracy under diverse conditions. Specifically, PiKd achieves a recall of 89.33% on average, outperforming the comparative methods by about 21.05%p to 41.25%p.

Keywords Piano keyboard detection · Deep Learning · OpenCV · Computer vision · Music transcription

1 Introduction

Object recognition is being actively studied based on the recent development of deep learning [1–8]. The recognition of various types of objects is being studied, including vehicles [9–12], lanes [13–15], license plates [16], and musical instruments [17–19]. In the present study, the detection of a piano keyboard among different musical instruments is investigated. Piano keyboard detection is a basic step in studies on music transcription, and an accurate detection of a piano keyboard is an important problem because it is linked to the accuracy of the music transcription [20–25].

Piano keyboard detection plays a foundational role in various music-related computer vision and video analysis tasks. Among these, automatic music transcription is the process of converting piano performances into symbolic music notation. This process relies critically on the accurate localization of the keyboard and its keys within video frames. Beyond music transcription, robust piano keyboard detection also supports applications such as automatic augmented reality for piano education and performance visualization. Therefore, enhancing the accuracy and robustness of piano keyboard detection, particularly in real-world video conditions, remains an important and challenging research problem.

Previous studies on piano keyboard detection have primarily focused on videos recorded in controlled environments [20–26]. These studies typically used footage captured from a fixed, top-down perspective, with consistent lighting and minimal background clutter. Under such conditions, high detection accuracy can be achieved using standard computer vision techniques [5, 9, 11]. However, real-world piano performance videos are often recorded in diverse environments, including concert halls, classrooms, and informal settings, and from varied camera angles, as illustrated in Fig. 1 [24, 27–30]. These variations introduce significant challenges, such as perspective distortion, partial occlusion by the performer's hands, and complex backgrounds, which substantially degrade the performance of conventional detection methods. As a result, detecting piano keyboards accurately in unconstrained video conditions remains an open problem.

Existing methods for object localization in unconstrained views, such as contour detection, shape-based segmentation, and geometric transformation-based coordinate alignment, typically rely on clear object boundaries, consistent lighting, and minimal occlusion. However, side-view piano performance videos frequently exhibit partial occlusions (e.g., hands, reflections), severe perspective distortions, and complex backgrounds, which substantially degrade the effectiveness of standard approaches. While recent advances in data fusion and

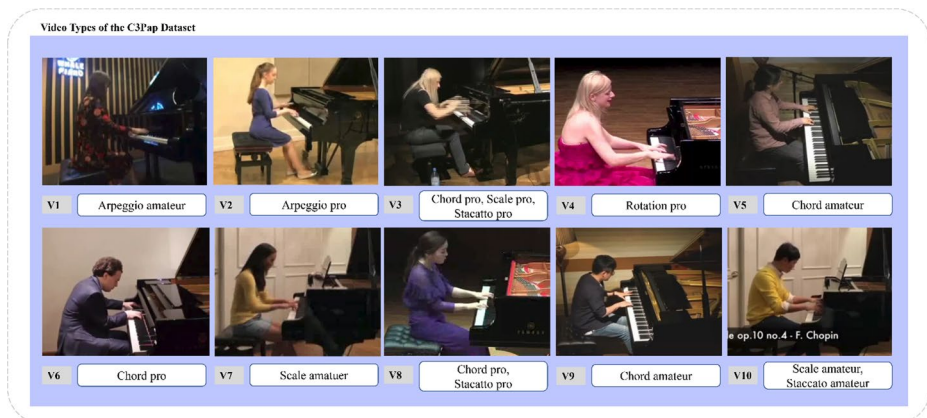


Fig. 1 Image data types used in this study

multi-frame image combination offer promising directions, they often assume static cameras or rigid object geometries, which do not fully apply to the highly variable context of piano performances.

To address these challenges, this study proposes a novel piano keyboard detection method (PiKd) designed for unconstrained real-world video conditions. Deep-learning-based detectors, such as Faster R-CNN or RT-DETR, excel at identifying objects but typically regress rectangular bounding boxes that cannot account for the severe trapezoidal perspective distortion found in side-view piano recordings. Conversely, conventional rule-based computer vision methods, such as Hough transforms or simple contour detection, fail to resolve dynamic occlusions caused by a performer's moving hands and are highly sensitive to the complex backgrounds and lighting shifts of unconstrained environments. Our PiKd framework solves these challenges by bridging the gap between robust semantic localization and precise geometric extraction. The pipeline first employs a YOLOv5-based object detector to localize the vicinity of the piano keyboard, establishing a scale-invariant Region of Interest (RoI) that filters out background noise. Once the RoI is established, our specialized refinement modules, our specialized refinement modules (ICM_PixInfo and PD_BiTh) operate as heuristic processing units to map specific coordinates. While modern detectors like YOLOv8 and RT-DETR offer high precision, YOLOv5 was selected for this pipeline due to its superior balance of accuracy and latency, which is critical for real-time applications. Our proposed method can be applied in various domains such as automatic music transcription, performance analysis, musical education tools, and gesture-based interaction systems. Specifically, the contributions of this study are as follows.

- We introduce ICM_PixInfo, a reference-based multi-frame fusion technique that enhances detection accuracy by combining partial pixel data. Unlike statistical median stacking, this method identifies the image with the fewest hand-colored pixels (HCC) as a structural anchor, removes detected hand regions, and synthesizes a “hand-free” keyboard image. This significantly reduces hand-related noise (Fig. 2) and improves the visibility of piano keys.
- We propose the Polygon Detection with Binary Threshold algorithm (PD_BiTh) to ex-

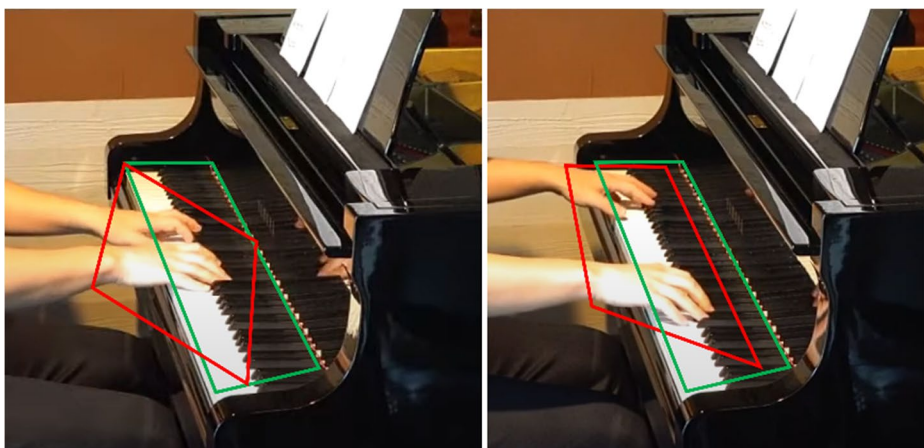


Fig. 2 Piano keyboard detection accuracy decreases due to hand noise. Here, the green box represents the ground truth of the piano keyboard, while the red bounding box indicates the result of the piano keyboard detection algorithm

tract piano key positions from combined images. This algorithm utilizes an automated, adaptive tuning loop to adjust the binarization threshold and color gamut (KCC), ensuring convergence even in low-contrast environments.

- To ensure geometric integrity, the Propriety Test of Degree of Expansion (PT_ExpDg) inspects expansion adequacy, while the Polygon Simplification Algorithm (PSAI) simplifies contours into precise four-sided quadrilaterals. Finally, the Substitution Algorithm (SubAI) ensures temporal consistency and computational efficiency by applying detected coordinates to every individual image in a sequence without redundant high-cost detection.
- We demonstrate the effectiveness of PiKd through extensive experiments using the C3Pap dataset, featuring a strict video-level split to prevent temporal leakage. We compare PiKd not only against conventional methods but also against modern deep learning baselines, including Faster R-CNN, YOLOv8, YOLOv10, and RT-DETR, evaluating performance across recall, precision, mAP, and computational overhead. Results show PiKd consistently achieves an average recall of 89.33%, outperforming comparative methods by approximately 21.05%p to 41.25%p.

The rest of the paper is organized as follows. In Sect. 2, previous related studies are introduced. In Sect. 3, the proposed method in this study is detailed. In Sect. 4, the qualitative and quantitative results of the experiments are analyzed. In Sect. 5, the results of the experiments are discussed. In Sect. 6, some concluding remarks are presented, and future research topics are suggested.

2 Related work

In this section, studies on piano keyboard detection using computer vision technology and studies related to object detection are introduced. Subsection 2.1 introduces studies on piano keyboard detection based on computer vision technologies. Then, we discuss the limitations of previous studies in Subsection 2.2.

2.1 Piano keyboard detection studies based on computer vision technology

Several studies have explored piano keyboard detection using computer vision techniques for different purposes. For example, Goodwin and Green [26] focused on beginner piano education and developed a system that detects the piano keyboard in webcam videos to teach finger positions. Their method applied binary thresholding, the Sobel operator, and the Hough transform, and demonstrated robustness to camera movement and partial occlusion of the keys. Lee et al. [21] proposed a virtual piano system that enables piano performance using only video input, without a physical instrument, with a key focus on piano key recognition. To detect the position of the piano keys in the input video, the study applies Hough Line Transform and binary thresholding to distinguish between white and black keys and to estimate the key boundaries. For detecting key presses, a spatiotemporal convolutional neural network (CNN) is employed to analyze hand movements and infer which keys are pressed and with what intensity. Suteparuk [20] proposed a method for piano keyboard detection and key tracking using techniques such as the Sobel filter, Hough transform, morphological operations, and region labeling. Beyond these classical approaches, recent advancements in deep learning have introduced powerful architectures for spatial localization, including two-stage detectors like Faster R-CNN [31], one-stage models such as the YOLO series (v5, v8, and v10) [32], and Transformer-based architectures like RT-DETR [33]. While these models offer high-level semantic feature learning, they typically regress rectangular bounding boxes which may struggle with the precise trapezoidal geometry of side-view piano keyboards.

Detecting the piano keyboard is a fundamental step for many downstream tasks, including music transcription. For example, Akbari and Cheng [23] proposed a real-time music transcription system that detects key presses after identifying the keyboard layout in videos filmed from above. Their method utilizes the Hough Line Transform and connected component labeling to achieve high accuracy and low latency, even under varying lighting conditions. Similarly, McCaffrey [24] developed a computer vision-based method for detecting the piano keyboard from side-view videos. The system identifies key locations, piano key values, and rhythmic patterns in real time. To achieve this, the authors employed techniques such as the Hough Line Transform, Canny edge detection, and Otsu's thresholding — standard image processing algorithms for edge and shape detection.

2.2 Limitations of previous studies

Previous piano keyboard detection studies have mostly used various computer vision technologies. However, previous methods have difficulty in accurately extracting and detecting the keyboard solely through an image processing technique when applying a dataset in which a number of different cases can occur. In particular, the shape of the piano profile is

a trapezoid, not a rectangle. Therefore, additional research is required to recognize the trapezoidal shape of a piano keyboard. Unlike existing methods, in this paper, a study on piano keyboard detection using a dataset of images captured in a real environment is conducted. In addition, the piano keyboard detection field has other problems to be solved such as hand noise. Therefore, a different approach is required.

3 Materials and methods

In this section, the proposed method is introduced. The overall flow of PiKd is described in Subsection 3.1, and the detailed process of the proposed method is described in Subsections 3.2 through 3.9.

3.1 Piano keyboard detection studies based on computer vision technology

The overall process of the proposed PiKd method is shown in Fig. 3. First, the process involves data acquisition step. Here, we use C3Pap, which is a dataset of piano playing videos. The videos in this dataset are of professional and amateur pianists playing various classical piano pieces [29]. In this study, all videos in the C3Pap dataset except those in black and white, videos where the piano keyboard cannot be seen due to the camera angle, and videos where the camera moves, were used. Next, a method of narrowing the range to the periphery of the piano keyboard was attempted for piano keyboard detection in Step (1)

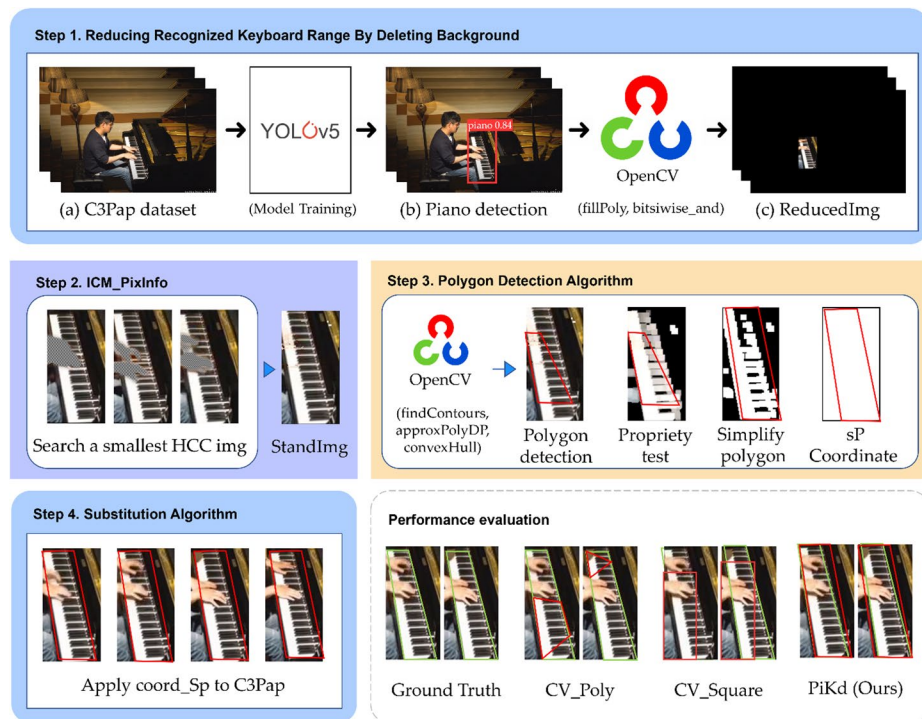


Fig. 3 Overall process of the proposed PiKd

To this end, the vicinity of the piano is detected using YOLOv5, which is an object detection method based on deep learning. YOLOv5 was trained using the custom C3Pap dataset, and the trained model was saved. Then, additional work is required to extract only the piano keyboard excluding the background. For this purpose, ICM_PixInfo, which removes hand noise from the image, is proposed in Step (2) Using various computer vision algorithms, polygons are detected from an image in which the hands were removed. As a result of detecting the polygon, the coordinates of the piano keyboard are extracted in Step (3) A substitution algorithm, which applies the extracted coordinates to other images, is used in Step (4) In the final step, the recall is measured by comparing the proposed method with the ground truth data.

3.2 C3Pap dataset

In this section, the data used in this study are introduced. The piano-playing video dataset C3Pap (Classic piano performance postures of amateur and professionals), which was originally introduced to our previous work [29]. The C3Pap dataset was developed to analyze piano-playing techniques and postures across varying skill levels. The dataset consists of 360 YouTube videos from 13 pianists (8 professionals and 5 amateurs) performing five techniques—chord, scale, arpeggio, rotation, and staccato—resulting in 11,850 annotated sequences extracted using OpenPose. Videos were filtered for quality, and skeleton data were normalized using Procrustes transformation and IQR-based outlier removal to ensure consistency.

Table 1 shows the data types among the C3Pap dataset used in this study. Videos were excluded if they were black and white, had extreme camera angles obscuring the keyboard, or involved camera movement, as these conditions hinder accurate piano keyboard detection. The action refers to the type of piano playing technique, the player level refers to the

Table 1 List of dataset selection among C3PAP

Action	Player level	Music sheet	Selected
Chord	Amateur	Prokofiev piano sonata no.2 mov.4	No
		Rachmaninov prelude op.3 no.2	Yes
	Pro	Chopin etude op.10 no.4	Yes
		Prokofiev piano sonata no.2 mo.4	Yes
Scale	Amateur	Rachmaninov prelude op.3 no.2	Yes
		Chopin etude op.10 no.4	Yes
		Chopin etude op.25 no.2	Yes
	Pro	Chopin etude op.10 no.4	Yes
		Chopin etude op.10 no.8	No
		Chopin etude op.25 no.2	No
Arpeggio	Amateur	Chopin etude op.25 no.12	Yes
		Chopin etude op.10 no.1	No
	Pro	Chopin etude op.25 no.12	No
		Chopin etude op.10 no.1	Yes
Rotation	Amateur	Liszt la campanella	No
	Pro	Liszt la campanella	Yes
Staccato	Amateur	Chopin etude op.10 no.4	Yes
		Prokofiev piano sonata no.2 mov.4	No
	Pro	Chopin etude op.10 no.4	Yes
		Prokofiev piano sonata no.2 mov.4	Yes

level of the player, which is divided into amateur and pro. The classes in Table 1 are the same as those labeled in C3Pap, and consist of five actions (chord, scale, arpeggio, rotation, and staccato) and two player levels (amateur and pro). If the class is a chord amateur, it means an amateur player who plays a chord. The music sheet refers to the name of the musical piece, and for the dataset selection, whether the data were used in this study is indicated by Yes or No.

3.3 Model training using YOLOv5

This study implements YOLOv5, a deep learning-based object detection algorithm that is widely recognized for its high speed and accuracy, making it well-suited for real-time applications such as piano keyboard detection. To justify this selection, we benchmarked YOLOv5 against Faster R-CNN, YOLOv8, YOLOv10, and RT-DETR. Our results reveal that while newer models like YOLOv8 and YOLOv10 offer slight improvements in mAP@50–95, YOLOv5 maintained the superior speed required for high-frequency video processing in real time. The YOLOv5 family includes models of varying sizes such as YOLOv5s, YOLOv5m, YOLOv5l, and YOLOv5x. While larger models such as YOLOv5x may offer marginally higher accuracy, they suffer from increased computational cost and slower inference speeds. Given that the objective of this study is to achieve efficient and accurate detection of piano keyboards in performance videos, we select YOLOv5s for its superior balance between speed and performance in real-time scenarios. The model was implemented using the official YOLOv5 framework provided by Ultralytics and trained on a set of images derived from the C3Pap dataset. The dataset was manually annotated with bounding boxes around piano keyboards using LabelImg, and the training procedures followed standard YOLOv5 configuration. Model training was conducted with default hyperparameters over five epochs, with model weights saved after each epoch to monitor performance progression. This setup enabled rapid training while maintaining reliable evaluation using key metrics such as precision, recall, and average precision. Figure 4 presents the visual results of piano keyboard predictions generated by the trained YOLOv5s model, demonstrating its effectiveness in identifying keyboard regions in real-world video frames. Based on these results, the proposed PiKd method demonstrates more precise and specific piano keyboard detection. The detailed process and components of the PiKd method are explained in the following subsections.

3.4 Reducing recognized keyboard range by deleting background

In this section, a method for reducing the range of piano keyboard detection is introduced. To increase the piano keyboard detection accuracy, the range is reduced from the total image to the vicinity of the keyboard. If the piano keyboard is to be detected from the entire image rather than reducing the range to the vicinity of the keyboard, the piano keyboard detection accuracy decreases because the background or person may be recognized as the keyboard. Thus, the pixels outside the box near the piano keys detected by YOLOv5 are painted black using the fillPoly function and bitwise_ operation of OpenCV [34].

The detailed process of this is shown in Algorithm 1. The input of the algorithm is C3PapImg, which indicates an image extracted from C3Pap. In addition, Pix indicates the number of pixels of the image. In line 1, the number of pixels is read from C3PapImg. In

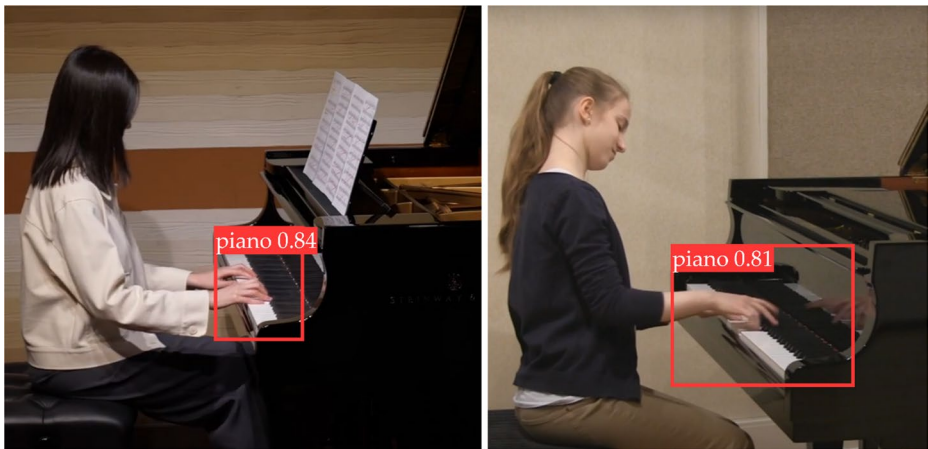


Fig. 4 Results of piano keyboard detection using YOLOv5 trained from a custom dataset

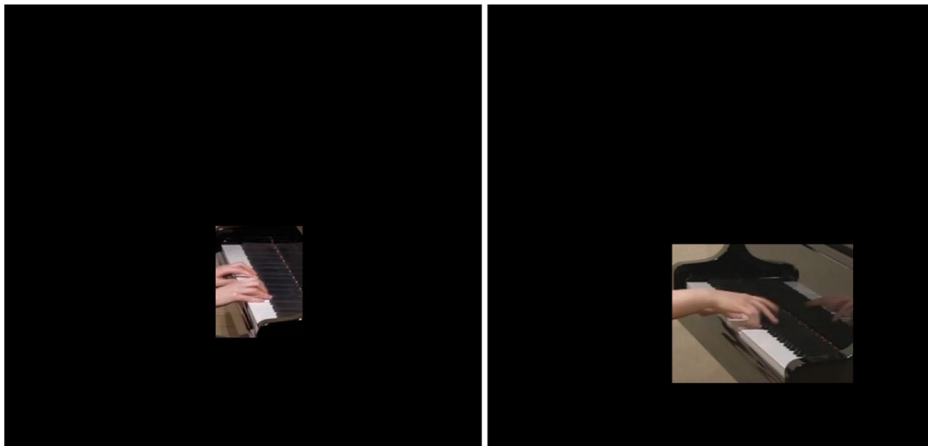


Fig. 5 Results of piano keyboard detection using YOLOv5 trained from a custom dataset

lines 2–3, the keyboard coordinates are painted white using the fillPoly function. The fillPoly function receives the coordinates of the image and polygon and then draws the polygon on the image. The remaining area excluding the part painted white is then painted black using the fillPoly function through a bitwise_and operation. As a result, the remaining pixels except for those inside the recognized box are displayed in black. In line 4, the image with a reduced range (ReducedImg) is returned as the output of the algorithm, as shown in Fig. 5.

Input*C3PapImg*: an image from C3Pap*Pix*: the number of pixels**Output***ReducedImg*: reduced image near the piano keys using YOLOv5

-
- 1: Read pixels information from *C3PapImg*
 - 2: **for** *i* **in** *Pix*:
 - 3: Paint excluded area near piano keys using *fillPoly* and *bitwise_and*
 - 4: **return** *ReducedImg*
-

Algorithm 1. Reducing the range of keyboard recognition

3.5 Image combining method using partial pixel information from multiple images (ICM_PixInfo)

In this section, the proposed ICM_PixInfo method is introduced. ICM_PixInfo is a method of synthesizing images using partial pixel information from multiple images. The accuracy of the piano keyboard detection decreases owing to the presence of hand noise. This study proposes a hand-removal method, which, based on the idea that the position of the piano is not changed when played, removes hands from multiple images and combines the pixel information.

The detailed process of ICM_PixInfo is shown in Algorithm 2. The inputs of the ICM_PixInfo algorithm are *ReducedImg*, which is an image with a reduced range; *Pix*, the pixel information of the image; and *HCC*, the hand color information. The output of the algorithm is *StandImg*, a combined image. First, in line 1, the number of pixels is calculated from *ReducedImg* for the loop. Lines 2 to 3 describe the process of finding the *StandImg* with the smallest hand color area among *ReducedImg*. The image with the smallest hand area is selected as the reference image by calculating the hand range among various images using *HCC*. The hand of the reference image is then replaced with a transparent color. In lines 4–9, the reference image *StandImg* is compared with the other images, *CompImg*. In lines 6–8, the reference image is compared with other images. If a color value exists at the corresponding position (within the transparent range at the reference position) in the compared image, the color value of the transparent range of the reference image is replaced with the color value of the compared image. The combined image *StandImg* is returned as the output of the algorithm.

Input

ReducedImg: reduced image near the piano keys using YOLOv5

Complmg: other images

Pix: the number of pixels in *ReducedImg*

HCC: hand color chart

Output

StandImg: combined image

-
- 1: Read pixels information from *ReducedImg*
 - 2: **for** i **in** *Pix*:
 - 3: Find the *StandImg* with the smallest *HCC* area among *ReducedImg*
 - 4: **for** i **in** *Pix*:
 - 5: Merge *Complmg* to *StandImg* with smallest *HCC* area
 - 6: **if** pixel color of *StandImg* = (255,255,255,0):
 - 7: **if** pixel color of *Complmg* != (255,255,255,0):
 - 8: Pixel color of *StandImg* := Pixel color of *Complmg*
 - 9: **return** *StandImg*
-

Algorithm 2. ICM_PixInfo

Figure 6 shows the resulting image of the proposed CM_PixInfo algorithm. The proposed method has the advantage of a low computation cost because of a lack of complex operations such as deep learning. Furthermore, despite its simplicity, the proposed method is effective for greatly improving performance.

3.6 Polygon detection algorithm (PD_BiTh)

In this section, PD_BiTh is introduced, which is a method for detecting a polygon in an image. The detailed process of PD_BiTh is shown in Algorithm 3. The inputs of this algorithm are the combined image, *StandImg*; th , which denotes the threshold for image binarization; c , which denotes how much to decrease from the threshold; and n , which denotes

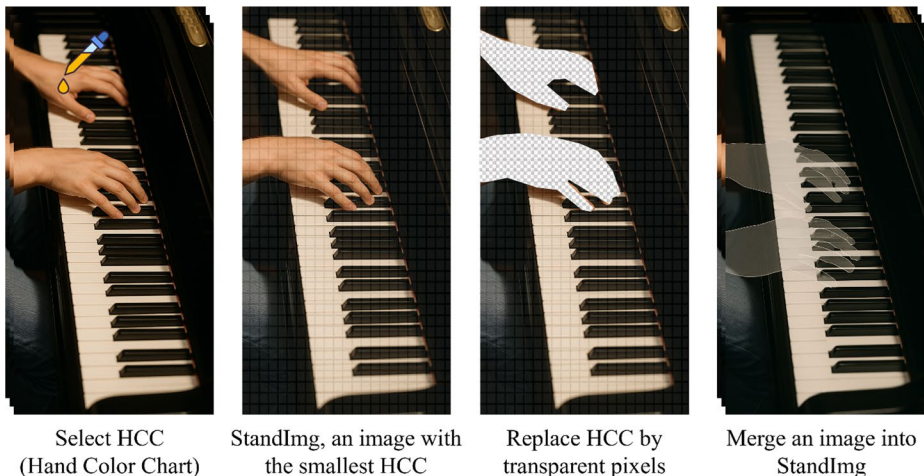


Fig. 6 Result image of ICM_PixInfo

the number of detected polygons. The last input value *KCC* denotes the keyboard color chart. The output of the algorithm is *P*, which indicates the finally detected polygon.

PD_BiTh represents a deterministic, convergent optimization process specifically tuned for keyboard geometry. Because the search space is finite ($th \in [0, 180]$) and the threshold *th* is monotonically decreased by step $c = 10$, convergence is guaranteed within a maximum of 18 iterations. In lines 1–2, *th* and *c* are initialized. The threshold *th* is initialized at 180 based on the high-contrast features of the C3Pap validation data. In line 3, detection of the polygon is attempted using the `findContours` function of OpenCV, which detects the polygon in `StandImg`. Lines 4–22 show the process of verifying whether the polygon was detected in the image.

The process differs depending on when no polygon is detected, when one polygon is detected, and when multiple polygons are detected. In lines 4–15, when no polygon is detected, detection is attempted again by expanding the keyboard color gamut and reducing *th*, which is used as the boundary value when the image is binarized. This automated tuning loop ensures that the framework remains robust across diverse lighting and perspective conditions where fixed thresholds would fail. In lines 16–21, when multiple polygons are detected, an inflation operation is conducted using *KCC* until the keyboard becomes a single image. Mathematically, for finite distances between disjoint components, there exists an inflation degree (*exp*) such that the dilated sets overlap, ensuring a single connected component where $n = 1$. In line 22, if one polygon is detected, a step for detecting a rectangle is carried out.

Input

StandImg: combined image

th: threshold value when binarizing an image

c: how much to decrease from *th*

n: the number of detected polygons

KCC: keyboard color chart

Output

P: final detected polygon

```

1: Initialize th = 180
2: Initialize c = 10
3: Detect polygon using findContours in StandImg
4: if n = 0:
5:   while n = 0 and th >= 0:
6:     th = th - c /*Decrease the threshold until a figure is detected*/
7:     Expand the keyboard color gamut using KCC
8:     if n >= 2:
9:       exp = 7 /*How much to inflate the polygon*/
10:      while n != 1:
11:        exp += 1 /* Expand until all shapes are connected as one */
12:      return P
13:    else:
14:      return P
15:  elif n != 1:
16:    exp = 7
17:    while n != 1:
18:      Expand the keyboard color gamut using KCC
19:      exp += 1
20:    return P
21: else:
22:  return P

```

Algorithm 3. PD_BiTh

3.7 Propriety test of degree of expansion (PT_ExpDg)

In this section, PT_ExpDg for testing the propriety of the degree of expansion of the detected polygon *P* is introduced. To prevent over-segmentation from excessive dilation, PT_ExpDg imposes a hard propriety boundary of *exp* = 20. The inputs of Algorithm 4 are *P*, which are the finally detected polygons of PT_ExpDg; the combined image *StandImg*; and *n*, the number of detected polygons. The output of PT_ExpDg is *rP*, which denotes the polygon detected again with a reduced degree of inflation. Each line is explained below. In line 1, *exp*, which denotes the propriety of the degree of expansion, is initialized to 20. In lines 2–11, the polygon is detected again by resetting the degree of inflation. In lines 2–5, if *exp* is 20 or less, it is determined that the inflation is adequate, and a polygon is drawn in *StandImg*. In line 4, *P* is substituted in *rP*, and this value is returned in line 5.

In lines 6–13, if the inflation was excessive, the degree of inflation is adjusted and the polygon detection is attempted again. In line 7, if exp is 20, it is determined that excessive inflation occurred and exp is initialized to 7. This default inflation value of 7 was tuned using the C3Pap dataset to provide a stable fallback that prevents divergence in low-contrast environments. Then, in line 8, inflation is attempted until all polygons are connected to a single polygon using the dilate function, which applies an inflation operation. Next, in lines 9–10, the inflated polygon is tested. If excessive inflation occurred, the degree of inflation is reset to the default value, and deterministic area selection is used to detect only the polygon with the largest area, ensuring the final output is the significant keyboard structure rather than background noise. In line 11, the finally detected polygon is drawn in *StandImg*. In line 12, P is substituted in rP , and this value is returned in line 13.

Input

P : final detected polygon

StandImg: combined image

n : the number of detected polygons

Output

rP : reduced polygon

```

1: Initialize  $exp = 20$ 
2: if  $exp \leq 20$ : /*If adequate expansion has been achieved*/
3:   Draw contours in StandImg
4:    $rP = P$ 
5:   return  $rP$ 
6: else:
7:   Initialize exponential  $exp = 7$  /*Reset degree inflation to default value of 7*/
8:   Dilate and find contours in StandImg
9:   for  $i$  in  $n$ :
10:    Detect only the polygon with the largest area among multiple polygons
11:   Draw contours in StandImg
12:    $rP = P$ 
13:   return  $rP$ 

```

Algorithm 4. PT_ExpDg

3.8 Polygon simplification algorithm (PSAI)

In this section, PSAI, which simplifies the detected polygon, is introduced. The inputs of Algorithm 5 are rP , which denotes the finally detected polygon in PT_ExpDg, and n , which denotes the number of detected polygons. The output of PSAI is sP , which denotes the simplified polygon. The detailed process of PSAI is as follows. In lines 1–4, the contours are approximated using the approxPolyDP function, and the shape of the finally detected polygon rP is simplified until it becomes a rectangle. In line 4, 4 is saved in s if the number of sides of the simplified polygon is 4. In lines 5–6, if s is 4, the detected polygon is first aligned clockwise using the order_points function. In lines 7–8, if the polygon is detected as a twisted shape even after the first alignment, the detection of a rectangular convex shape is attempted using the convexHull function. In lines 9–10, if no rectangle is extracted as a

result of simplification or it fails to detect a rectangular convex shape, the smallest rectangle that is circumscribed in the detected shape is drawn using the `minAreaRect` function. In line 11, the simplified rectangle sP is returned.

Input

rP : final reduced polygon

n : the number of detected polygons

Output

sP : simplified polygon

```

1: for  $i$  in  $n$ :
2:   Simplify polygon  $rP$  using approxPolyDP
3:    $sP = rP$ 
4:    $s = \text{len}(sP) / \text{the number of polygon sides} /$ 
5:   if  $s = 4$ :
6:     Sort  $sP$  using order_points
7:   if isContourConvex( $sP$ ) is FALSE:
8:     Find square using ConvexHull
9:   else:
10:    Draw smallest rectangle circumscribed in shape using minAreaRect
11:  return  $sP$ 

```

Algorithm 5. PSAl

3.9 Substitution algorithm (SubAl)

In this section, the substitution algorithm SubAl, which applies the detected piano key positions to a different piano image, is introduced. The inputs of SubAl in Algorithm 6 are `coord_Sp`, which denotes the coordinates of the simplified rectangle Sp ; `C3PapImg`, which denotes the C3Pap image; and `m` which denotes the number of C3PapImgs. The output of SubAl is `P_Value`, which denotes the predicted coordinates of the keyboard location of the proposed method. In lines 1–2, `coord_Sp` is applied to all C3PapImgs and is saved in `P_Value`. In line 3, `P_Value` is returned.

Input

$Coord_sP$: coordinate of simplified polygon

$C3PapImg$: all C3Pap image

m : the number of $C3PapImg$

Output

P_Value : Predicted coordinate value of keyboard location

```

1: for  $i$  in  $m$ :
2:    $P\_Value += coord\_sP$  to  $C3PapImg$ 
3:  return  $P\_Value$ 

```

Algorithm 6. SubAl

4 Results

In this chapter, an experiment is conducted. Subsection 4.1 describes the ground truth collection method. Subsection 4.2 explains the performance evaluation metrics. Subsection 4.3 introduces comparative methods. Finally, Sect. 4.4 provides the experiment results.

4.1 Experimental setup

The experiments were conducted on a device with an Intel Core i9-9900 K CPU, 64GB of DDR4 RAM, and an NVIDIA GeForce RTX 2080 GPU with 8GB of VRAM. All models were implemented using the Python programming language within a PyTorch-based environment. Specifically, the YOLO variants were developed using the Ultralytics framework, while the Faster R-CNN and DETR (Detection Transformer) architectures were implemented using the Torchvision and official PyTorch libraries. This hardware configuration ensured a consistent environment for measuring both the accuracy and the real-time processing capabilities of the different methods.

4.2 Dataset

The experiments utilize the C3Pap dataset, a collection of video sequences capturing various piano performance techniques. The ground truth annotations were generated using Labelme, an open-source graphical image annotation tool. During the labeling process, polygons are manually drawn around the piano keyboard in each frame, and the resulting metadata is exported in JSON format. When the keyboard image overlaps with the images of human hands, or other objects, labeling is conducted by focusing on the keyboard. We tried to make the shape of the keyboard into a parallelogram as much as possible, and when impossible, we tried to make it into a trapezoidal shape. This allows for precise spatial localization of the keyboard regardless of the camera angle or the presence of the performer's hands.

The dataset is partitioned into training, validation, and test sets using a video-level split. This approach ensures that frames from the same video sequence do not appear in multiple sets, thereby evaluating the model's performance on entirely unseen data. As shown in Table 2, the dataset is categorized by five distinct piano levels: Chord, Scale, Arpeggio, Rotation, and Staccato, further divided by player level (Amateur and Pro). The dataset encompasses diverse environmental conditions, including varying lighting intensities, different camera perspectives, and a range of background (overhead vs. angled), settings. The training set consists of 1,563 images, while the validation and test sets contain 521 images each, maintaining a balanced ratio (i.e., 6:2:2) for consistent evaluation.

Table 2 Dataset details by number of frames

Action Level	Train			Validation			Test		
	Amateur	Pro	Total	Amateur	Pro	Total	Amateur	Pro	Total
Chord	145	194	339	51	62	113	44	69	113
Scale	133	208	341	62	79	141	45	73	118
Arpeggio	145	74	219	47	21	68	48	25	73
Rotation	78	166	244	18	32	50	24	42	66
Staccato	207	213	420	80	69	149	73	78	151
Total	708	855	1,563	258	263	521	234	287	521

4.3 Evaluation metrics

The performance was evaluated using the Intersection over Union (IoU), Precision, Recall, and Average Precision at IoU 0.5 (mAP@0.5). These metrics are particularly important for piano keyboard detection because they quantify both the accuracy and completeness of the model's predictions. High precision and recall confirm that all visible keyboard regions are detected. A high IoU ensures that the predicted keyboard boundaries align closely with the actual keyboard. High AP@0.5 provides robust indicators of model performance across different thresholds and scenarios.

The IoU is defined as the area of overlap between the ground truth bounding box B_{gt} and the predicted bounding box B_p , divided by area of their union as shown in Eq. 1.

$$IoU = \frac{B_{gt} \cap B_p}{B_{gt} \cup B_p} \quad (1)$$

Based on the IoU threshold (commonly set at 0.5), true positives (TP) are identified. Recall measures the model's ability to detect all relevant instances and is defined by Eq. 2.

$$Recall = \frac{TP}{TP + FN} \quad (2)$$

where FN is the number of false negatives. Precision indicates the proportion of correctly predicted positive instances and is defined as Eq. 3.

$$Precision = \frac{TP}{TP + FP} \quad (3)$$

where FP is the number of false positives. AP@0.5 (Average Precision at IoU 0.5) is computed by integrating the precision-recall curve when $IoU \geq 0.5$, representing the area under that curve.

4.4 Comparative methods

The proposed PiKd framework is a pipeline that leverages the strengths of both data-driven deep learning and rule-based image processing. To evaluate its performance, the comparative analysis is divided into two distinct stages: the initial detection of the keyboard vicinity and the subsequent refinement of its specific coordinates.

4.4.1 Deep learning initial detection stage

The first stage is responsible for the robust localization of the piano keyboard within a raw image frame. Because conventional image processing struggles in datasets with complex backgrounds, we employ a Deep Learning backbone to reduce the recognized range to the vicinity of the keyboard. To justify our architectural choices, we benchmarked several state-of-the-art detectors [35–37]:

- Two-Stage Detector: Faster R-CNN was evaluated for its high-precision region-proposal architecture.
- One-Stage Detectors: We compared YOLOv5 (our chosen backbone) with newer iterations, YOLOv8 and YOLOv10, to assess improvements in feature learning.
- Transformer-Based Detector: RT-DETR was included to evaluate the effectiveness of global self-attention mechanisms in keyboard spatial localization.

Table 3 shows the selected hyperparameter settings for the piano keyboard localization methods.

4.4.2 Rule-based refinement methods

The conventional methods detect a piano keyboard using various image processing methods and apply algorithms to recognize the piano keyboard in images extracted from videos filmed in limited environments using videos taken above the piano keyboard. Hence, the methods could detect the piano keyboard using a relatively simple image processing technique. However, this study aims to solve the problem of extracting the keyboard from the side view of the piano and uses a dataset in which different variables exist. Therefore, the keyboard detection accuracy was significantly low when the conventional simple image processing algorithm was used. In particular, the accuracy dropped significantly unless the vicinity of the piano keyboard was detected using YOLOv5. Therefore, for a fair comparison, the vicinity of the piano keyboard was detected using YOLOv5, and the performances of the image processing algorithms from previous studies were compared with the performance of the algorithm proposed herein.

In addition to conventional research algorithms, additional algorithms need to be developed for comparing the algorithms used in conventional studies with the method proposed in this study. This is because a conventional algorithm detects the top and bottom lines of the piano keyboard, and it is difficult to compare the accuracy in such a case. Hence, although previous studies detect lines using computer vision algorithms, in this study, line detection is changed to polygon detection for a performance comparison. Moreover, unlike the previous studies, the dataset used in this study is a dataset of filmed side views.

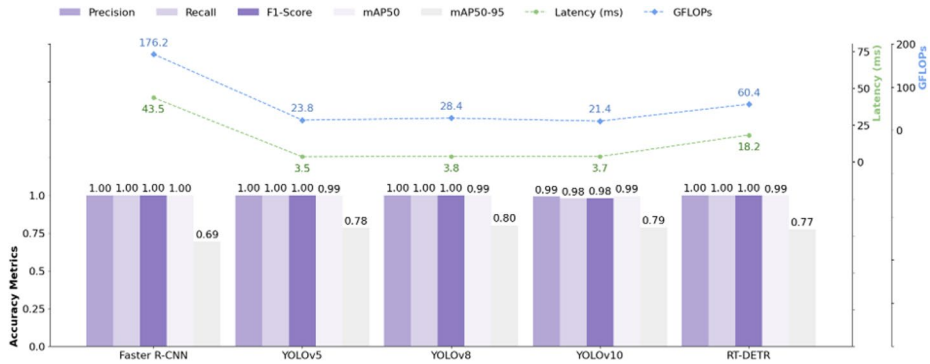
The detailed algorithm of CV_Poly is as follows. First, CV_Poly reduces the range to the vicinity of the piano using YOLOv5. Next, the polygon in the image is detected using an image processing technique. If multiple polygons are detected, they are inflated until all polygons are connected into a single polygon. Next, whether the inflation was adequately

Table 3 Hyperparameter settings for the piano keyboard localization methods

Parameter	Faster R-CNN	YOLOv5 / v8 / v10	RT-DETR
Epochs	100	100	100
Patience	15	15	15
Batch Size	4	16	8
Input Size	640 × 640	640 × 640	640 × 640
Optimizer	SGD	SGD / AdamW	AdamW
Initial LR	0.01	0.01	0.0001
Weight Decay	0.0001	0.0005	0.0001
Augmentation	random rotation, scaling, brightness, contrast, blur, and mosaic		

Table 4 Comparative methods and proposed PIKD

Method	First step	Second step
CV_Poly	YOLOv5	Polygon detection algorithm
CV_Square	YOLOv5	Square detection algorithm
PiKd (ours)	YOLOv5	ICM_PixInfo+Polygon detection algorithm+SubAI

**Fig. 7** Performance comparison of state-of-the-art detectors for piano keyboard localization

performed is determined. If the inflation was excessive, it is initialized to the default inflation value, and lines are detected instead of polygons. If one polygon is detected, the detected polygon is simplified. If a rectangle is detected as a result of the simplification, the rectangle is drawn in the image; otherwise, the smallest rectangle that is circumscribed on the detected polygon is drawn.

The details of the CV_Square algorithm are as follows. Unlike CV_Poly, CV_Square only extracts squares instead of polygons, and the extracted square is expanded to the keyboard range. Table 4 shows the comparison of the proposed PiKd and the comparative methods.

4.5 Experimental results

4.5.1 Comparative performance analysis of backbone detectors

Figure 7 illustrates a clear trade-off between architectural complexity and inference efficiency. While all models achieved nearly perfect scores in Precision, Recall, and F1-score (~ 1.00), the mAP50-95 metric serves as the primary differentiator for localization quality. YOLOv8 attained the highest precision (mAP50-95 = 0.80), followed closely by YOLOv10 (0.79) and YOLOv5 (0.78), indicating that the YOLO-based architectures are exceptionally robust at regressing piano keyboard boundaries. In terms of computational overhead, Faster R-CNN proved unsuitable for real-time deployment due to its high latency (43.5 ms) and substantial GFLOPs (176.2). Minimizing these computational requirements is critical for real-time applications like automatic music transcription. Conversely, YOLOv10 demonstrated the most streamlined profile with only 21.4 GFLOPs, while YOLOv5 maintained the lowest latency at 3.5 ms. This 3.5 ms latency is nearly 12 times faster than Faster R-CNN and significantly faster than RT-DETR. Despite its advanced self-attention mechanism, RT-

DETR fell behind the YOLO iterations in both mAP (0.77) and speed (18.2 ms), suggesting that for the specific task of keyboard localization, the computational cost of transformers does not yield superior accuracy compared to optimized CNN-based one-stage detectors.

Figure 8 shows the analysis of the loss curves over 25 epochs revealing distinct learning behaviors across the evaluated methods. The Faster R-CNN validation loss exhibited significant fluctuations and a high degree of variance, signaling instability in the region proposal phase for this specific dataset. In contrast, the YOLO series (v5, v8, and v10) showed consistent, smooth convergence with a narrowing gap between training and validation loss, which confirms the effectiveness of their data augmentation and optimization strategies. The minimal gap between training and validation loss serves as mathematical proof that the video-level split effectively prevents overfitting and ensures the models have generalized well to unseen data. YOLOv10 and YOLOv8, in particular, reached a stable equilibrium faster than YOLOv5, reflecting improvements in their respective backbone architectures. RT-DETR displayed the most stable training curve with the lowest overall loss values; however, this stability did not translate to the highest mAP, as shown in the performance chart. Overall, the consistent convergence of the YOLO family, paired with their superior mAP and low-latency profiles, justifies the selection of a YOLOv5-based backbone as the most viable solution for the robust localization of piano keyboards in real-time environments.

Figure 9 shows the examples of qualitative validation results and internal feature map activations of the selected backbone YOLOv5. As demonstrated in our validation batches, the model consistently achieves a high confidence score of 0.9 across diverse environments. Figure 9(a) illustrates that the model's spatial focus remains strictly locked on the piano keyboard, even under challenging conditions. To provide a deeper look into the network's "learned spatial focus," we have visualized the intermediate feature map activations. As shown in Fig. 9(b), the network effectively suppresses background noise (such as the performer, the piano lid, and the room decor). High-intensity activations (bright regions in several channels) are concentrated specifically on the linear edges and periodic white-black key transitions. This confirms that the model is not "guessing" based on the presence of a person but is accurately extracting the unique textures and spatial frequencies of the piano keyboard itself.

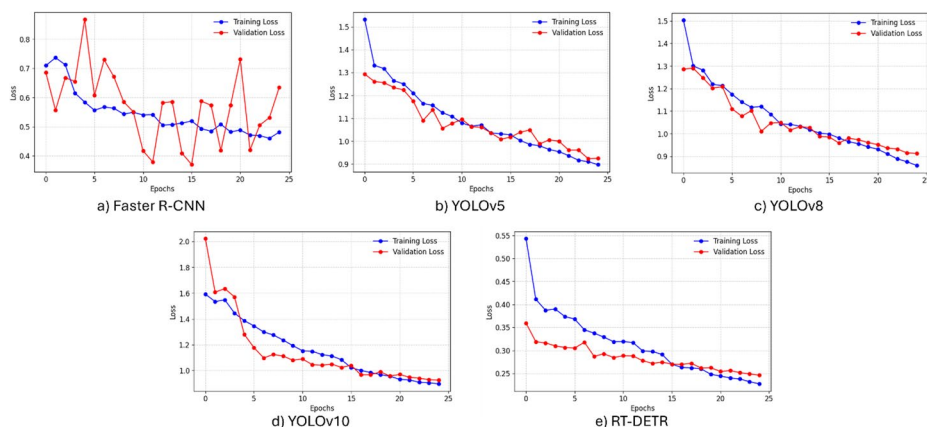
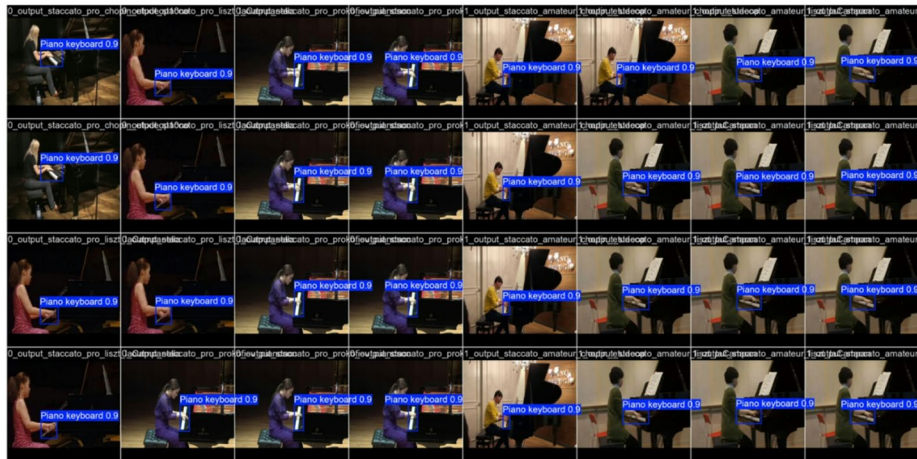
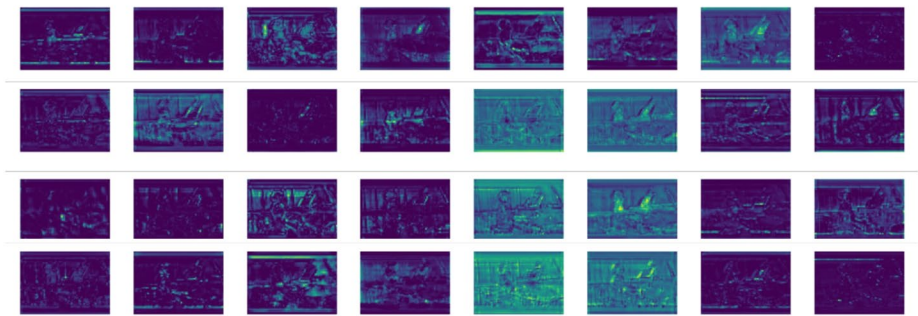


Fig. 8 Training and validation loss curves across evaluated detection methods



a) Qualitative validation results



b) Internal feature map activations

Fig. 9 Examples of (a) qualitative validation results and (b) internal feature map activations of the YOLOv5 model. (a) Qualitative validation results (b) Internal feature map activations

4.5.2 Quantitative results of conventional methods and proposed PIKD

In this subsection, we compare the performance of the proposed method with that of conventional approaches. We measured the recall of each method while varying the required area of overlap between the predicted and ground truth regions, setting thresholds at 20%, 40%, 60%, and 80%. These overlap thresholds serve as a quantitative proxy for evaluating robustness under varying levels of hand occlusion. As shown in Fig. 10, we evaluate the comparative methods using five actions: chord, scale, arpeggio, rotation, and staccato, as well as two player levels: amateur and pro. The highest recall values were observed at the 20% overlap threshold and recall consistently decreased as the overlap requirement increased. This trend indicates that all methods struggled more with precise localization as the evaluation criterion became stricter.

Among the evaluated methods, the proposed PiKd approach demonstrated the best overall performance across all thresholds, followed by CV_Square and then CV_Poly. Specifi-

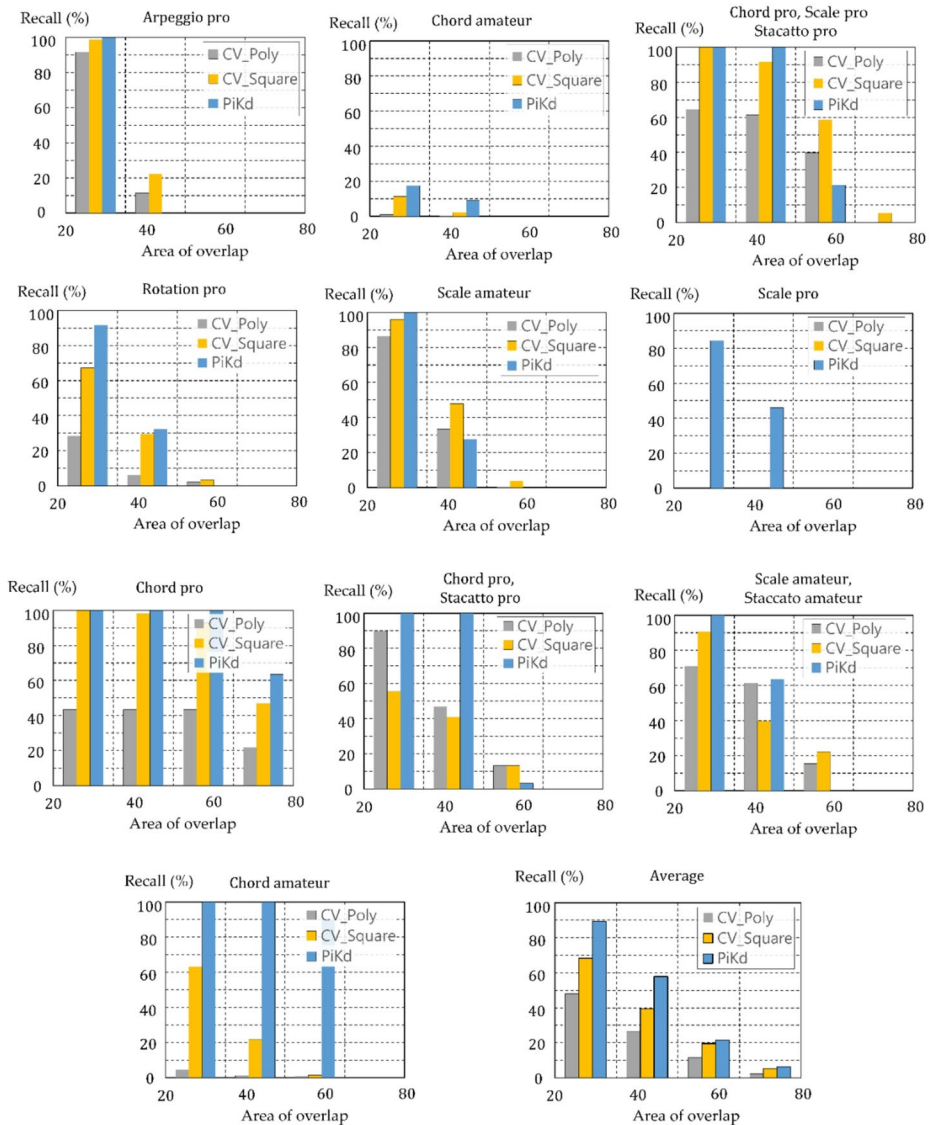


Fig. 10 Experimental results of PiKd

cally, on average, PiKd outperforms CV_Square and CV_Poly by 21.05%p and 41.25%p, respectively. The 41.25%p improvement over CV_Poly, which utilizes similar geometry extraction but lacks multi-frame synthesis, directly quantifies the critical contribution of ICM_PixInfo in addressing dynamic hand occlusions. These results highlight the robustness of PiKd in detecting the piano keyboard region accurately, even under varying degrees of overlap requirements, and confirm its superiority over conventional detection methods in terms of recall. This performance gain is consistent across ten distinct video categories and both skill levels, providing large-scale empirical evidence of the framework's stability.

The statistical significance of the reported performance improvement, ranging from 21.05%p to 41.25%p, is supported by its consistency across the high-diversity C3Pap testing set. This evaluation was conducted on a testing set comprising 521 images derived from 360 different video sequences, ensuring that the results are representative of a broad range of real-world scenarios rather than a small, biased sample. Furthermore, the superiority of the PiKd framework remains stable across all ten distinct video categories and both amateur and pro player levels. As illustrated in the average recall analysis, PiKd maintains a dominant margin over both CV_Poly and CV_Square across every overlap threshold, from the lenient 20% to the strict 80%. This large-scale empirical evidence demonstrates that the proposed hybrid architecture provides a substantial and stable improvement over conventional image-processing baselines under unconstrained conditions.

5 Discussion

5.1 Qualitative analysis of conventional methods and proposed PiKd

In this subsection, we analyze the experimental results by visually comparing the outputs of each method. Figure 11 illustrates representative result examples, clearly highlighting the differences in detection performance among the methods. The (a) and (b) of Fig. 11 correspond to the detection results of CV_Poly and CV_Square, respectively. CV_Poly recognized a very small area of the left-hand tip as the piano keyboard. CV_Square, on the other hand, detected the piano keyboard including the hands. In contrast, the proposed PiKd, Fig. 11(c), method detected the piano keyboard as relatively similar to the ground truth. These visual comparisons support quantitative results and demonstrate that PiKd achieves more accurate and precise detection than conventional methods.

Figure 12 also shows the result screens indicating that the proposed PiKd method exhibited a higher detection rate than the comparative methods. The (a) and (b) of Fig. 12 correspond to the detection results of CV_Poly and CV_Square, respectively. Similar to previous observations, CV_Poly again identified only a minimal portion of the region—specifically,

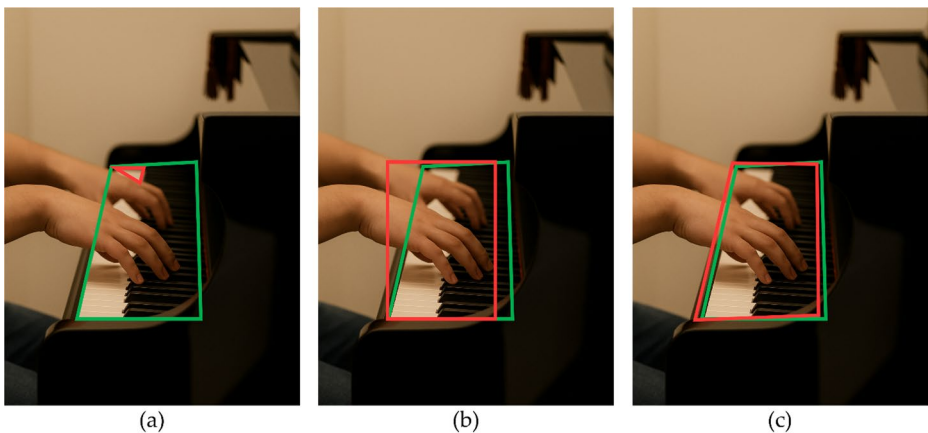


Fig. 11 Case 1 : Illustration of experimental results of conventional methods and proposed PiKd: (a) CV_Poly, (b) CV_Square, (c) PiKd

a small area near the right-hand tip, leading to a significantly underestimated detection. Meanwhile, CV_Square overestimated the target region by classifying almost the entire visible area as the piano keyboard, including the pianist's hands and surrounding space. By contrast, the proposed PiKd, Fig. 12(c), detected the piano keyboard as similar to the ground truth. These results visually confirm that PiKd provides a better balance between precision and completeness.

5.2 Limitation of PiKd

Despite its overall effectiveness, the PiKd method has certain limitations, as illustrated in Fig. 13. In some cases, accurate detection of the piano keyboard decreased due to issues with the ICM_PixInfo component, which is responsible for pixel-level information processing. As shown in Fig. 13(a), the piano keyboard detection tended to be poor when one hand is not visible on the screen. In Fig. 13 (b-c), while this study introduced improvements that partially mitigated these challenges, occasional cases remained where only the white keys were successfully detected, while the black keys were missed. These findings suggest that while PiKd outperforms conventional methods in general, its performance may still be sensitive to hand positions, lighting conditions, and color contrast between the performer and the keyboard.

6 Conclusions

This study proposed a new piano keyboard detection method that combines the deep-learning-based object recognition technology YOLOv5 with computer vision technology. Previous studies detect the piano keyboard in videos filmed above the piano within limited environments. By contrast, this study proposed a piano keyboard detection method using a dataset of videos showing actual piano playing in side-view.

The process of the proposed PiKd is composed of data acquisition, model training based on YOLOv5 to reduce the range of piano keyboard detection, and a step for applying an

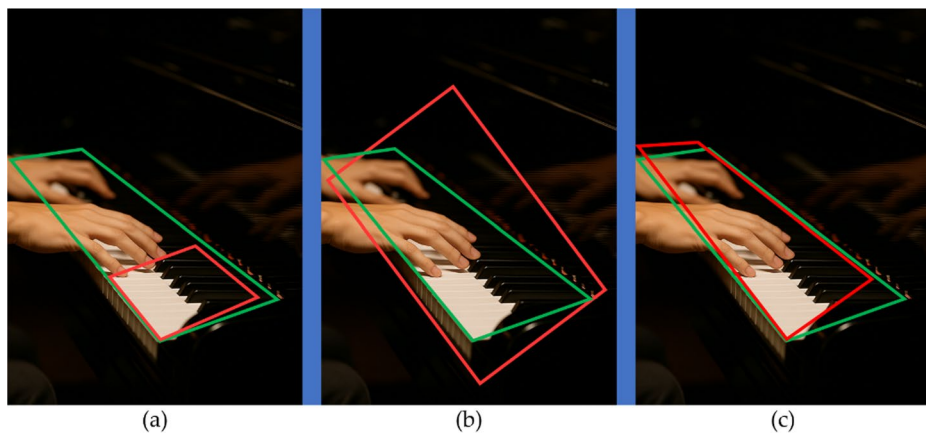


Fig. 12 Case 2 : Illustration of experimental results of conventional methods and proposed PiKd: (a) CV_Poly, (b) CV_Square, (c) PiKd

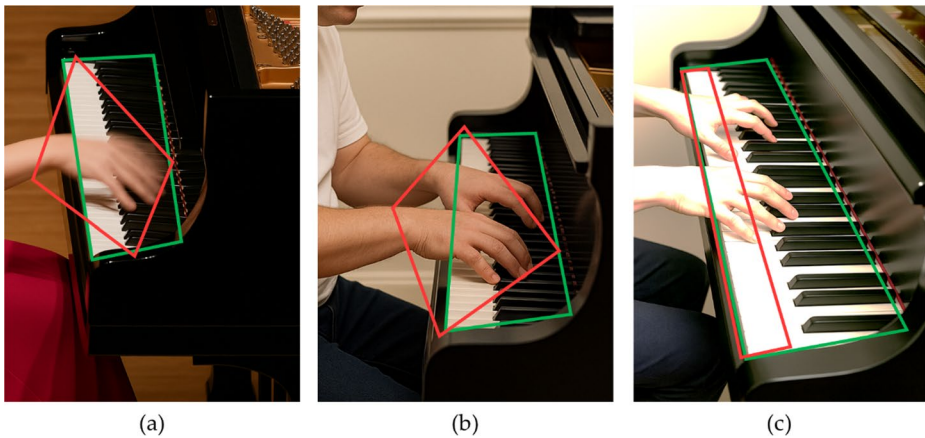


Fig. 13 Case 3 : Illustration of keyboard detection failure of PiKd : (a) one hand is not visible on the screen, (b) finger part was displayed in white and affected the black keys, and (c) part of the wrist moved far beyond the piano keyboard

additional algorithm to recognize trapezoidal keyboards in side-view videos. First, for data acquisition, a classical piano video dataset, called C3Pap, was collected. For the model training, ground truth data for the keyboard position are collected using Labelme, and the model for recognizing the keyboard based on this data is saved. In the final piano keyboard detection step, the vicinity of the piano keyboard is detected using the trained model based on YOLOv5. Next, to improve the piano keyboard detection accuracy, ICM_PixInfo is applied, which is an image combination method for the removal of hand noise proposed in this study. To remove hand noise, which decreases the accuracy of piano keyboard detection in the images, ICM_PixInfo combines the pixel information of multiple images from which the hands have been removed. After applying ICM_PixInfo, the piano keyboard is detected using the polygon detection algorithm based on computer vision technology. Finally, the substitution algorithm is applied, and the accuracy (recall) is measured. As a result of comparing the performance of the proposed method with the results of the conventional methods, the proposed PiKd showed a higher performance. Specifically, our proposed PiKd achieves a recall of 90%, outperforming the comparative methods CV_Poly and CV_Square by about 21.05%p to 41.25%p in the best case.

The proposed PiKd method enables robust piano keyboard detection in real-world settings, extending its usefulness beyond basic object recognition. It can be effectively applied in fields such as automatic music transcription, performance analysis, and music education tools. For example, accurate keyboard localization allows for aligning hand movements with specific keys, improving the transcription of performances and providing feedback in educational applications. This highlights PiKd's potential as a core component in intelligent music analysis systems.

To address the limitations observed in the current study, future work will focus on enhancing the robustness of PiKd in complex visual scenarios. Specifically, we plan to improve the ICM_PixInfo component by incorporating adaptive color segmentation techniques that can better distinguish between the performer's skin tones and the piano keys under varying lighting conditions. Moreover, expanding the dataset to include more diverse

hand positions, lighting setups, and camera angles will support better generalization and reduce failure cases. Lastly, we will conduct a music transcription study using the findings of this study.

Author contribution Tserenpurev Chuluunsaikhan: Conceptualization, Investigation, Writing – review & editing, Visualization. So-Hyun Park: Conceptualization, Investigation, Methodology, Data curation, Formal analysis, Writing – original draft, Visualization, Funding acquisition, Project administration, Supervision.

Funding This work was supported by the National Research Foundation of Korea (NRF) grant funded by the Korea Government (MSIT) (No. 2021R1C1C2004282).

Data availability Data available on request from the authors.

Declarations

Competing interests The authors declare that they have no competing interests.

References

1. Paredes HF, Suryawati E, Zilvan V, Ramdan A, Kusumo RB, Heryana A, Yuwana RS, Krisnandi D, Subekti A, Fauziah F, Rahadi VP (2020) Plant diseases detection with low resolution data using nested skip connections. *J Big Data* 7:57
2. Go S-E, Kim J-H, Chuluunsaikhan T, Choi W-S, Choi S-H, Nasridinov A (2024) Unified Generative Data Augmentation for Efficient Solar Panel Soiling Localization. *Electronics* 13:4859
3. Choi J-H, Kim J-H, Nasridinov A, Kim Y-S (2024) Three-dimensional atrous inception module for crowd behavior classification. *Sci Rep* 14:14390
4. Ka E, Go S, Kwak M, Kim J-H, Nasridinov A (2024) Solar Power Generation Forecasting via Multimodal Feature Fusion (student abstract). *Proc AAAI Conf Artif Intell* 38:23530–23532
5. Lin YY, Liu TL, Fuh CS (2004) Fast object detection with occlusions. In *Computer Vision – ECCV 2004. 8th European Conference on Computer Vision, Prague, Czech Republic, May 11–14, Proceedings, Part I*; Pajdla, T., Matas, J., Eds.; Lecture Notes in Computer Science; Springer: Berlin/Heidelberg, Germany, 2004; Volume 3021, pp. 402–413
6. Xu J, Wang H, Xu M, Yang F, Zhou Y, Yang X (2021) Feature-enhanced occlusion perception object detection for smart cities. *Wirel Commun Mob Comput*
7. Xiang Y, Savarese S, Object detection by 3D aspectlets and occlusion reasoning, In *Proceedings of the IEEE International Conference on Computer Vision (ICCV), Workshops (2013)* pp. 530–537
8. Wang A, Sun Y, Kortylewski A, Yuille A (2020) Robust object detection under occlusion with context-aware compositional nets, In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR)*, pp. 12642–12651
9. Sivaraman S, Trivedi MM (2010) A general active-learning framework for on-road vehicle recognition and tracking. *IEEE Trans Intell Transp Syst* 11:267–276
10. Zhang L, Tao H, Sun Z, Yi W (2025) Multi-Strategy Enhancement of YOLOv8n Monitoring Method for Personnel and Vehicles in Mine Air Door Scenarios. *Sensors* 25:3128
11. Kato T, Ninomiya Y, Masaki I (2002) Preceding vehicle recognition based on learning from sample images. *IEEE Trans Intell Transp Syst* 3:252–260
12. Foresti GL, Murino V, Regazzoni C (1999) Vehicle recognition and tracking from road image sequences. *IEEE Trans Veh Technol* 48:301–318
13. Kim J, Lee M (2014) Robust lane detection based on convolutional neural network and random sample consensus. *Lect Notes Comput Sci* 454–461
14. Li T, Xiong X, Zhang Y, Fan X, Zhang Y, Huang H, Hu D, He M, Liu Z (2025) RE-YOLOv5: Enhancing Occluded Road Object Detection via Visual Receptive Field Improvements. *Sensors* 25:2518
15. Ojala R, Vepsäläinen J, Tammi K (2022) Motion detection and classification: Ultra-fast road user detection. *J Big Data*, 9, 28
16. Kim T-G, Yun B-J, Kim T-H, Lee J-Y, Park K-H, Jeong Y, Kim HD (2021) Recognition of Vehicle License Plates Based on Image Processing. *Appl Sci* 11:6292

17. Solanki A, Pandey S (2019) Music instrument recognition using deep convolutional neural networks. *Int J Inf* 14:1659–1668
18. Shreevathsa PK, Harshith M, Rao A (2020) Music instrument recognition using machine learning algorithms, In *Proceedings of the International Conference on Computation, Automation and Knowledge Management (ICCAKM)*, pp. 161–166
19. Krishna AG, Sreenivas TV (2004) Music instrument recognition: From isolated notes to solo phrases, In *Proceedings of the IEEE International Conference on Acoustics, Speech, and Signal Processing*, vol. 4, pp. IV-IV
20. Suteparuk P (2014) Detection of piano keys pressed in video, Dept. of Comput. Sci., Stanford Univ., Stanford, CA, USA, Tech. Rep
21. Lee J, Doosti B, Gu Y, Cartledge D, Crandall D, Raphael C (2019) Observing pianist accuracy and form with Computer Vision, In *Proceedings of the IEEE Winter Conference on Applications of Computer Vision (WACV)*, pp. 1505–1513
22. Kang S, Kim J, Yoon SE (2019) Virtual piano using computer vision. *arXiv preprint arXiv:1910.12539*
23. Akbari M, Cheng H (2015) Real-time piano music transcription based on Computer Vision. *IEEE Trans Multimedia* 17:2113–2121
24. McCaffrey R (2017) Piano music transcription based on computer vision, M.S. thesis, Dept. Computer Science., Dublin Univ., Dublin, Ireland
25. Li J, Xu W, Cao Y, Liu W, Cheng W (2020) Robust piano music transcription based on computer vision, In *Proceedings of the High Performance Computing and Cluster Technologies Conference and 2020 & 3rd International Conference on Big Data and Artificial Intelligence*. pp. 92–97
26. Goodwin A, Green R (2013) Key detection for a virtual piano teacher, In *Proceedings of the 28th International Conference on Image and Vision Computing New Zealand (IVCNZ 2013)*, pp. 282–287
27. Park SH, Nasridinov A, Park YH (2015) A Kinect-based piano education system for correction of pianist posture. *Asia Life Sci* 12:571–586
28. Park S-H, Ihm S-Y, Nasridinov A, Park Y-H (2019) A feasibility test on preventing prmds based on Deep Learning. *Proc AAAI Conf Artif Intell* 33:10005–10006
29. Park S-H, Park Y-H (2020) Audio-Visual Tensor Fusion Network for Piano Player Posture Classification. *Appl Sci* 10:6857
30. Beacon JF, Comeau G, Payeur P, Russell D (2017) Assessing the suitability of Kinect for measuring the impact of a week-long feldenkrais method workshop on pianists' posture and movement. *JMTE* 10:51–72
31. Rani S, Ghai D, Kumar S (2022) Object detection and recognition using contour based edge detection and fast R-CNN. *Multimedia Tools Appl* 81:42183–42207. <https://doi.org/10.1007/s11042-021-11446-2>
32. Vijayakumar A, Vairavasundaram S (2024) YOLO-based Object Detection Models: A review and its applications. *Multimedia Tools Appl* 83:83535–83574. <https://doi.org/10.1007/s11042-024-18872-y>
33. Zhao Y, Lv W, Xu S, Wei J, Wang G, Dang Q et al (2024) Detsr beat Yolos on real-time object detection, 2024 IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR). 16965–16974. <https://doi.org/10.1109/cvpr52733.2024.01605>
34. OpenCV – Open Computer Vision Library <https://opencv.org/> (2025)
35. Ultralytics/yolov5 (2025) YOLOv5 in PyTorch <https://github.com/ultralytics/yolov5>
36. Magalhães SA, Castro L, Moreira G, dos Santos FN, Cunha M, Dias J, Moreira AP (2021) Evaluating the single-shot multibox detector and YOLO deep learning models for the detection of tomatoes in a greenhouse. *Sensors* 21, 3569
37. Wang Y, Yu P (2021) A Fast Intrusion Detection Method for High-Speed Railway Clearance Based on Low-Cost Embedded GPUs. *Sensors* 21:7279

Publisher's note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.

Authors and Affiliations

Tserenpurev Chuluunsaikhan¹ · So-Hyun Park² 

✉ So-Hyun Park
sohyunpark@dongguk.ac.kr

¹ Bigdata Research Institute, Chungbuk National University, Cheongju 28644, Korea

² Department of Computer Science and Engineering, Dongguk University, Gyeongju 38066, Korea